

Phase - 1 Documentation

Category Balance for Failure Clustering

Generating 100 runs using the code [generate.py](#) the estimated failures are given below to ensure no single category dominate or occur more than 40% times, we have calculated the overall distribution percentage given by the formula

Distribution Percentage = $\sum (\text{failure probability}(\text{primary or secondary}) * \text{est. failures})$

Expected failures per test over 100 runs, and failure type distribution:

Test Name	Primary Failure Type	Secondary Failure Type	Est. failures in 100 runs
TC_Login_SSORedirect	timeout (70%)	element (30%)	~35
TC_Login_MFAVerification	timeout (70%)	assertion (30%)	~30
TC_Dashboard_LoadWidget	element (80%)	timeout (20%)	~50
TC_Dashboard_RefreshData	assertion (60%)	data (40%)	~55
TC_User_BulkImport	data (70%)	assertion (30%)	~65
TC_User_RoleAssignment	assertion (65%)	data (35%)	~80
TC_User_BatchExport	data (65%)	element (35%)	~75
TC_Login_OAuthCallback	timeout (70%)	element (30%)	~70

****Estimated category totals across ~460 total failures:****

Category	Estimated count	Distribution Percentage
timeout	$\sim 35 \times 0.7 + \sim 30 \times 0.7 + \sim 50 \times 0.2 + \sim 70 \times 0.7 = \sim 115$	~25%
element	$\sim 35 \times 0.3 + \sim 50 \times 0.8 + \sim 75 \times 0.35 + \sim 70 \times 0.3 = \sim 107$	~23%
assertion	$\sim 30 \times 0.3 + \sim 55 \times 0.6 + \sim 65 \times 0.3 + \sim 80 \times 0.65 = \sim 110$	~24%
data	$\sim 55 \times 0.4 + \sim 65 \times 0.7 + \sim 75 \times 0.65 + \sim 80 \times 0.35 = \sim 145$	~31%

No single category exceeds 40%. Distribution is balanced enough for clustering to work. The `data` category is slightly higher due to the high-volume consistently-failing tests; acceptable within spec.

Current code ensures even distribution in failure category, and also uses the probabilities for primary and secondary reason of failure, though the instance of occurrence of the primary or secondary (similar to est. number of failures) deviates from original probability distribution sometimes.

Modification of Pass Rate values

Since, there are 12 stable tests in the current test definitions out of 20 in total given in the design_doc.md the lowest pass rate(except anomaly) can only be 60%. In order to implement the pass rate curve we have slightly modified the values keeping the trend still intact.

```
## Pass Rate Curve (Original Values)
```

```
| Runs | Pass Rate | Story |
|-----|-----|-----|
| 1-25 | 60-70% | Early instability |
| 26-35 | 65-55% | Gradual decline |
| **36** | **25%** | ⚠ ANOMALY – spike failure |
| **37** | **28%** | ⚠ ANOMALY – still broken |
| 38-45 | 55-50% | Partial recovery |
| 46-75 | 55-80% | Recovery sprint |
| 76-100 | 82-95% | Stable strong finish |
```

```
## Pass Rate Curve (Modified Values)
```

```
| Runs | Pass Rate | Story |
|-----|-----|-----|
| 1-25 | 70-80% | Early instability |
| 26-35 | 72-65% | Gradual decline |
| **36** | **25%** | ⚠ ANOMALY – spike failure |
| **37** | **28%** | ⚠ ANOMALY – still broken |
| 38-45 | 65-60% | Partial recovery |
| 46-75 | 65-80% | Recovery sprint |
| 76-100 | 82-95% | Stable strong finish |
```

```
---
```

Flaky test failures are overall slightly more clustered together than random because of force failure is used for pass rate curve. Yet the flaky flips (change from pass to fail or vice versa) would remain detectable clearly.

Bidirectional Pass Rate Correction

After all 20 tests roll their natural outcomes, the actual failure count rarely matches the target exactly. The correction closes that gap.

```
target_failures = round(total_tests * (1 - target_pass_rate))
natural_failures = sum(1 for _, outcome in results if not outcome)
extra_failures_needed = target_failures - natural_failures
```

`extra_failures_needed` is a signed integer that drives both directions:

Positive — too many passes, need more failures:

```
candidates = [
    i for i, (test, outcome) in enumerate(results)
    if outcome and test[4] != "stable"
]
force_fail_indices = set(rng.sample(candidates, min(extra_failures_needed, len(candidates))))
```

Only non-stable passing tests are candidates. Stable tests are excluded because their only valid failure mode is `environment`-type, which is reserved for anomaly runs — forcing one outside that context corrupts the failure type distribution.

Negative — too many failures, need to recover some:

```
# flaky first, consistently_failing only if still needed
for pool in (candidates_flaky, candidates_cf):
    still_needed = extra_passes_needed - len(chosen)
    if still_needed <= 0:
        break
    chosen += rng.sample(pool, min(still_needed, len(pool)))
```

Flaky tests are preferred because a flaky test passing is a normal event. Consistently-failing tests are only touched if the flaky pool is exhausted. This keeps the correction narratively conservative.

Both index sets are resolved before the XML loop runs. `build_test_xml` receives `force_fail` or `force_pass` as a flag and applies the override before duration, timestamps, or any XML is constructed — so a force-corrected test is structurally identical to a naturally-occurring one.

The two branches are mutually exclusive (`if/elif`), so no test can be in both sets in the same run.

Anomaly Runs

The expected sudden spike in failure and drop of pass rate to just 25-28% is achieved by Stable test cases failing. Stable tests are assigned an environment failure type used exclusively on anomaly runs. All other runs, stable tests do not fail and this config entry is not invoked.

Element Tree is used for XML builder

Flaky test Variation, Duration Pattern, Failure Message Variance and Everything else is implemented as mentioned in the `generator_spec.md`

Dependency Model - (Creative Addition)

In a real test suite, tests don't fail in isolation. If a login test fails, every test that requires a logged-in session is at elevated risk — not guaranteed to fail, but more likely to. The `DEPENDENCIES` dict encodes exactly this relationship.

```
DEPENDENCIES = {
    "TC_Dashboard_FilterByDate": {"deps": ["TC_Login_ValidCredentials"], "weight": 0.4},
    "TC_Dashboard_Pagination": {"deps": ["TC_Login_ValidCredentials"], "weight": 0.4},
    "TC_Dashboard_ExportChart": {"deps": ["TC_Dashboard_FilterByDate"], "weight": 0.5},
    "TC_Dashboard_LoadWidget": {"deps": ["TC_Login_ValidCredentials"], "weight": 0.6},
    "TC_User_BulkImport": {"deps": ["TC_User_CreateAccount"], "weight": 0.5},
}
```

Each entry says: *"this test depends on these upstream tests, and if they failed, propagate risk with this weight."*

The dependency graph looks like this:

```
TC_Login_ValidCredentials
├── TC_Dashboard_FilterByDate (weight 0.4)
│   └── TC_Dashboard_ExportChart (weight 0.5)
├── TC_Dashboard_Pagination (weight 0.4)
└── TC_Dashboard_LoadWidget (weight 0.6)

TC_User_CreateAccount
└── TC_User_BulkImport (weight 0.5)
```

How It Works — The Multiplicative Risk Model

The model runs during the **first pass** over `TESTS` (before bidirectional correction). Because `TESTS` is ordered and the loop goes top to bottom, upstream outcomes are already known by the time a downstream test is evaluated. They're tracked in `natural_outcomes` as the loop progresses.

For each test that has a dependency entry, it checks how many of its dependencies already failed this run. If any did, it adjusts the `fail_prob` using this formula:

```
effective_fail_prob = 1 - (1 - base_fail_prob) * (1 - weight × failed_dep_count)
```

Then clamps to 0.95 — no test is ever guaranteed to fail.

Worked example — `TC_Dashboard_FilterByDate` (base `fail_prob` = 0.0, weight = 0.4):

- Normal run, `TC_Login_ValidCredentials` passed → `failed_dep_count = 0` → model doesn't activate → `fail_prob` stays `0.0` → test passes as usual
- Anomaly run, `TC_Login_ValidCredentials` failed → `failed_dep_count = 1`:

```
effective = 1 - (1 - 0.0) * (1 - 0.4 × 1)
           = 1 - 1.0 * 0.6
           = 0.4
```

So a previously rock-solid stable test now has a 40% chance of failing — purely because its upstream dependency went down. That's realistic: if you can't log in, the dashboard filter test is likely broken too.

Chained example — `TC_Dashboard_ExportChart` depends on `TC_Dashboard_FilterByDate` :

If `FilterByDate` fails (itself possibly because login failed), `ExportChart` then gets its own `fail_prob` raised by weight 0.5. Failures can cascade two levels deep through the graph.

Why It Matters for the Dataset

Without dependencies, each test's outcome is statistically independent. A clustering or anomaly detection model trained on that data would learn that failures are random and uncorrelated — which is not how real CI pipelines behave.

With dependencies, the generated data has correlated failure patterns that reflect real-world causal structure:

- On a bad login run, you'll see `ValidCredentials` fail and then `FilterByDate`, `Pagination`, `LoadWidget` all fail together in the same run — a realistic cascade
- `ExportChart` failures will co-occur with `FilterByDate` failures more often than chance
- `BulkImport` failures will correlate with `CreateAccount` failures

This gives an ML model something real to find: not just "these tests fail a lot" but "when test A fails, tests B and C tend to fail in the same run." That's a meaningful signal for root cause clustering, which is the whole point of the dataset.

Weight	Interpretation
0.4	Moderate coupling — upstream failure raises downstream risk by 40 percentage points from baseline
0.5	Medium-strong — dashboard export is meaningfully dependent on filter working
0.6	Strong coupling — <code>LoadWidget</code> is tightly tied to login; if auth is broken, the widget almost certainly can't load

The 0.95 clamp ensures that even with multiple failed deps and a high base `fail_prob`, the test still has a 5% chance of passing — preserving the probabilistic nature of the data rather than creating hard deterministic failures.